

SOR

Assignment 4 : Report

> How I estimated distance ?

For calculating the distance, I added a second subscription of camera in my modified code using `self.depth_subscription` and then it listens to the `camera/depth_image`. So when there is a depth frame the `depth_callback` function is called which stores the output in `self.depth_frame`. Then inside `run_yolo` when Yolo detects the target object it calculates the centre of the object by the formula $cx = (x1 + x2) // 2$ and $cy = (y1 + y2) // 2$. This calculated value is then used to check the depth of the object by the formula `self.depth_frame[cy,cx]`. And then it gives the distance. So, in short depth camera is being used to calculate the distance using required functions and formulas.

> How my search behaviour works ?

There is a `search_behaviour` function in my modified code and this function is being called at the end of every `run_yolo` part. Then the target is checked whether it's in the current frame or not by `target_found` which uses `len(detections) > 0`. Then inside the function (`search_behaviour`) three conditions are checked first it checks if the person has entered the target object or not then if it passes it checks if the target is already visible i.e within the camera range and then when this statement returns it checks if the bot is actually at the specified distance or not. If any of these conditions are not passed (True) implies the robot can't see the target so then the robot is moved or rotated by using the twist message and then the robot moves slowly and when the robot locates the target the twist function stops working.

> How your robot decides when to move and stop ?

So I have used a function named `hunt_target` and it's inside `run_yolo` (which detects the target) and it is called when a target is detected and it's distance is measured . This function takes three inputs `cx` , `frame_width` , `distance`. So , first of all the `self.target_reached` is run to check if the bot is already at the required distance from the object i.e it's `True` in this case the bot doesn't move as it's already at the required location . But if it returns `False` then the function calculated $\text{error} = cx - \text{frame_centre}$ | and this error tells how far and in which direction left or right is the target from the centre of camera frame . Then this error is used to determine the angular velocity(`angular.z`) of bot by the formula $-0.5 * (\text{error}/\text{frame_width})$ which steers the robot left or right toward the target . At the same time the linear velocity (`linear.x`) is set which moves the bot towards the object after detecting it . This continues till the bot reaches the maximum distance from the object so that it doesn't collide with it and at this point both linear (`linear.x`) and angular(`angular.z`) velocities are then nullified to 0 (Zero) and the robot stops . Also to stop the robot to move again if the loop keeps running the `self.target_reached` is set to zero.